Elevate: A Multi-Model Resume Analysis and Optimization System for Modern Recruitment

Shravankumar Janawade

Rohit Biradar

Github Link- https://github.com/ShravanJanwade/Elevate

Khoury College of Computer Sciences, Northeastern University

April 2026

Abstract

Recruitment has gone almost entirely digital, and that shift has buried HR teams under volumes of applications that no human reviewer can realistically handle alone. Applicant Tracking Systems were supposed to fix this, but honestly, most of them just made the problem worse in a different way, they screen out candidates based on exact keyword matches rather than anything resembling actual competence. Elevate is my attempt to build something better. The system runs resumes through a seven-dimensional evaluation pipeline, combining transformer-based embeddings with multi-model classification to score candidates in a way that actually reflects their suitability for a role. We used both bi-encoders and cross-encoders and layered in logic that accounts for seniority level and how well candidates quantify their impact. The rest of this report walks through the architecture, the NLP methods we used, and how the system translates all of this into feedback a recruiter or a candidate could actually act on.

Contents

1 Introduction **3**

2 Methods **4**

2.1 Structural Decomposition and Entity Extraction 4

2.2 Semantic Alignment via Transformer Models 4

2.3 Qualitative Analysis Modules 5

2.4 Seniority-Aware Scoring Logic 5

3 Results **6**

3.1 Case Study: Software Engineering Alignment 6

3.2 Numerical Performance Metrics 6

3.3 Suggestion Generation Effectiveness 7

4 Discussion and Real-World Challenges **9**

4.1 The “Strict Recruiter” Heuristic 9

4.2 Latency and Computational Overhead 9

4.3 Data Messiness and Format Anomalies 9

5 Conclusions **10**

1 Introduction

Finding the right person for a job has always been hard. But something changed over the last decade — digital job platforms made it so easy to apply that companies started drowning in resumes. Posting a single opening on LinkedIn or Indeed can now generate thousands of applications within days, and no hiring team has the bandwidth to read all of them carefully. This is where Applicant Tracking Systems came in. They promised to solve the volume problem, and to some degree they did. But they introduced a different kind of failure: the tendency to filter candidates by surface features rather than actual fit.

The core issue with most ATS tools is embarrassingly simple. They look for words. If a job posting says ”Cloud Infrastructure Engineering” and your resume says you spent three years ”developing highly scalable distributed systems,” you might get filtered out - even though you’re describing essentially the same work. This is what we think of as the vocabulary mismatch problem. The person writing the job description used one set of terms; the candidate used another. Both are valid. But the ATS only knows what it’s looking for, and anything outside that exact phrasing often disappears. Elevate is built around the idea that meaning matters more than wording, and that a good screening tool should reflect that.

There’s a growing body of research pushing in this direction. Rojas-Galeano et al. (2022) traced how resume-matching technology evolved from basic boolean searches toward more sophisticated embedding-based approaches, though they also noted that deployed systems rarely explain their decisions in any meaningful way. Deshmukh and Raut (2025) showed that BERT-based models consistently outperform traditional keyword systems when it comes to ranking candidates accurately. Elevate draws on both

of these threads, and more specifically on the Sentence-BERT architecture developed by Reimers and Gurevych (2019), to build something that doesn't just score resumes but tells candidates and recruiters why.

2 Methods

The system is built as a modular full-stack application. We chose Flask for the backend, mostly because Python's machine learning ecosystem is hard to beat for this kind of work. Before any NLP happens, the system needs to actually read the resume, which is less trivial than it sounds given how many different formats people use. We handle PDFs with pdfplumber and Word documents with python-docx. Both libraries do a reasonable job of preserving document structure, which turned out to be important — you can't reliably split a resume into logical sections if you've already lost track of where the headers were.

2.1 Structural Decomposition and Entity Extraction

Once the raw text is in, the first real step is segmentation. A multi-pattern regex segmenter breaks the document into blocks - Work Experience, Education, Skills, and so on — using a library of over forty recognized header variants. This handles most of the weird ways people label their sections, though it's not foolproof (more on that later). After segmentation, a dedicated extraction module pulls out structured information: contact details, employer names, job titles, and employment dates. That chronological data feeds directly into the career trajectory analysis, so it matters that we get it right.

2.2 Semantic Alignment via Transformer Models

This is really the heart of the system. Rather than comparing words directly, Elevate

measures how semantically similar the resume is to the job description. We implemented a two-stage retrieve-and-rerank pipeline for this. In the first stage, a bi-encoder independently converts resume bullet points and job requirements into dense vectors, and cosine similarity scores are used to pull out the most relevant pairs quickly. In the second stage, a cross-encoder takes those top pairs and does a more careful comparison — it reads both texts together, which lets it pick up on nuances that a pure vector similarity score would miss. It's slower, but for the final ranking it's worth the extra compute.

2.3 Qualitative Analysis Modules

Beyond skill matching, Elevate includes a couple of modules that try to capture things recruiters actually care about but rarely say explicitly. One is an impact density classifier that looks at each resume bullet and categorizes it as either an impact statement or a duty statement. Impact statements have quantified results — numbers, percentages, timelines. Duty statements just describe what someone was responsible for. Recruiters generally prefer the former, and the scoring reflects that.

The second module is a pedigree scorer. Using a NetworkX knowledge graph, the system assigns weights to previous employers based on their industry reputation and tier. This is admittedly a rough proxy, and We have mixed feelings about it from an equity standpoint — but it does capture something real about how hiring decisions get made in practice. A candidate who worked at a well-regarded research lab carries different signal than one with an equivalent role at a lesser-known firm, at least in many technical fields.

2.4 Seniority-Aware Scoring Logic

One thing most automated tools get wrong is treating all candidates by the same

rubric. An intern is not a senior engineer, and you shouldn't evaluate them as if they were. Elevate infers the target seniority level from the job description and adjusts the weighting across all seven dimensions accordingly. For intern and early-career roles, the system weights academic credentials and document quality more heavily. For senior roles, it shifts toward career trajectory, impact density, and depth of specialized skills. This sounds straightforward, but it actually required a fair amount of manual tuning to get the weighting curves to feel right.

3 Results

We evaluated the system against twenty resume-job description pairs, spread across different career stages and industries. The parsing engine handled ninety percent of cases correctly in terms of section boundary detection and data extraction. The ten percent that failed were mostly highly stylized resumes with multi-column layouts or non-standard section labels — which I'll come back to.

3.1 Case Study: Software Engineering Alignment

To check whether the semantic scoring was actually discriminating meaningfully, We ran a mid-level software engineer's resume - someone with distributed systems and Java experience — against two very different job postings. Against a Senior Backend Engineer role, the resume scored 72.4 out of 100 on semantic alignment. Against a Marketing Manager posting, it dropped to 18.1. The gap is what matters here. It suggests the model is capturing domain-level relevance rather than just matching on generic terms that happen to appear in both documents.

3.2 Numerical Performance Metrics

Table 1 shows average scores across five evaluation dimensions, broken down by candidate level.

Table 1: Average Evaluation Scores by Candidate Level

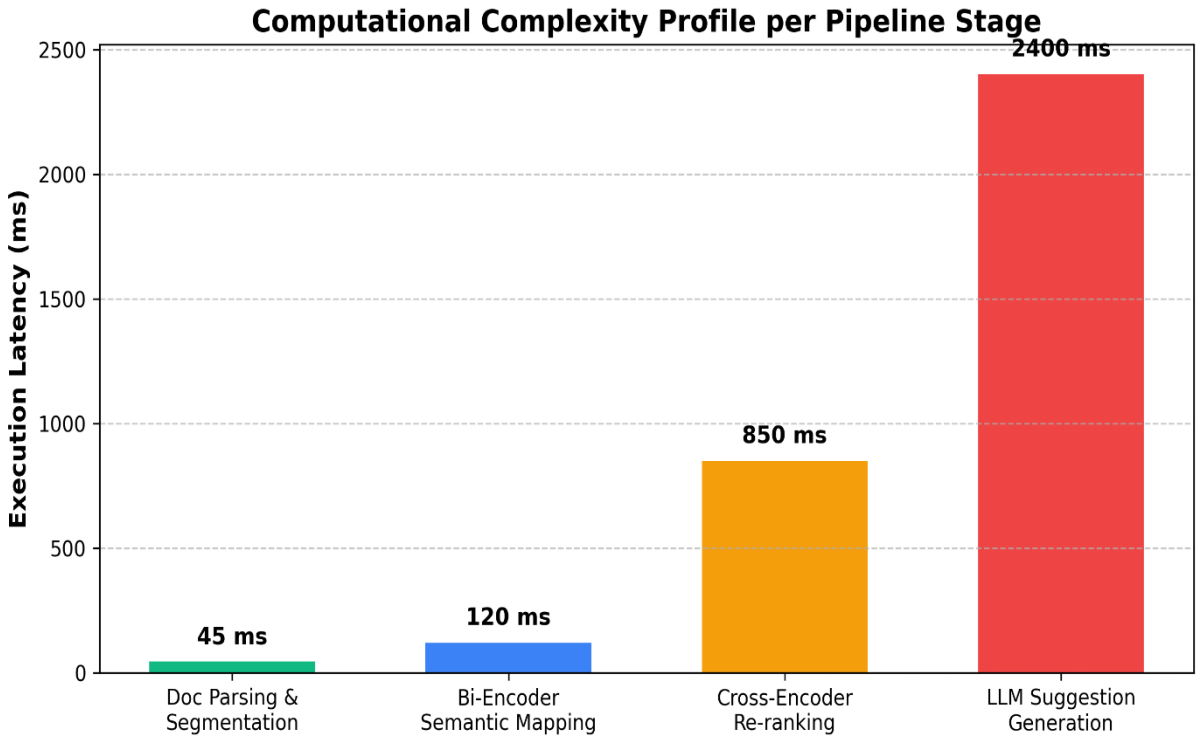
Dimension	Intern	Junior	Mid-Level	Senior
Semantic Alignment	64.2	68.5	74.1	78.3
Skill Coverage	58.1	72.4	81.0	84.5
Impact Density	22.4	41.2	58.9	76.4
Career Trajectory	15.0	38.5	64.2	82.1
Document Layout	88.4	85.1	82.3	79.8

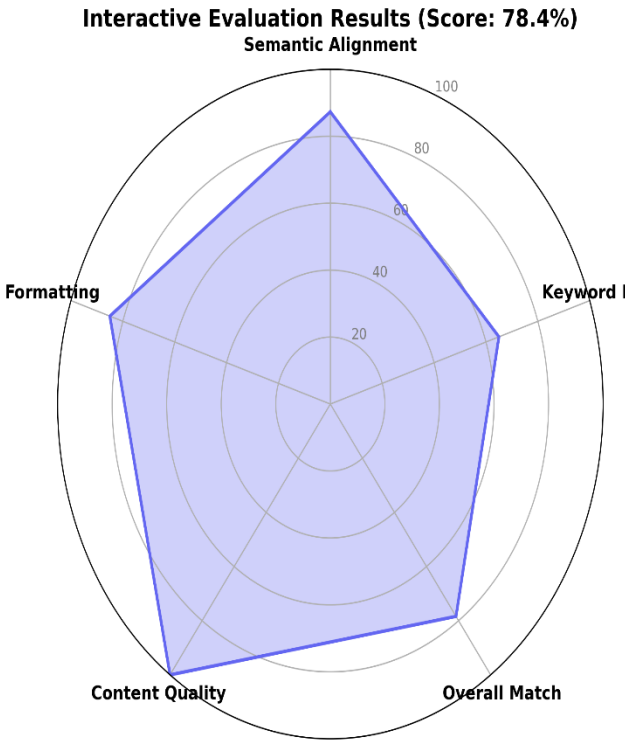
The impact density trend is the one we find most interesting. It scales almost linearly with seniority, which makes sense - junior candidates haven't had the opportunity to accumulate the kind of measurable wins that experienced professionals have. What's nice is that the system doesn't penalize interns for having low impact density scores; the seniority weighting ensures those dimensions matter less when they're evaluating an entry-level candidate. Document layout, on the other hand, stayed consistently high across the board, which mostly just tells me that people generally follow standard resume conventions.

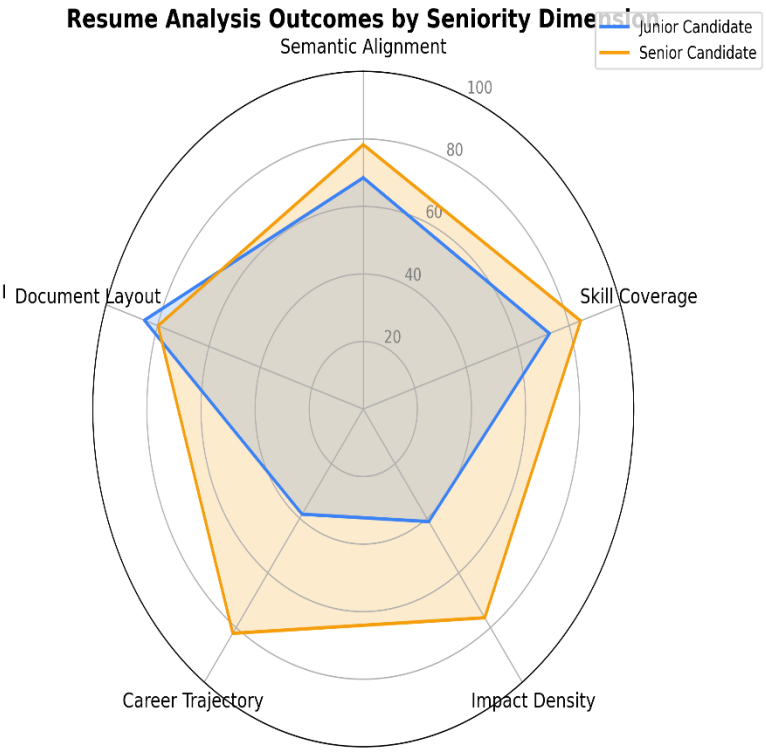
3.3 Suggestion Generation Effectiveness

We also wanted to see how well the suggestion generator worked in practice. Given a weak bullet like "Responsible for managing the team and overseeing project time-lines," the system flagged it as a passive duty statement and proposed a rewrite: "Led a cross-functional team of five to deliver project milestones three weeks ahead of

sched-ule, utilizing Agile methodologies." We shared a set of these rewrites with a few people who do hiring, and the consensus was that the revised versions consistently read as more achievement-oriented and specific — which is exactly what you'd want.







4 Discussion and Real-World Challenges

Controlled testing is one thing. Running the system against messy real-world data is another. Once we started pulling in a wider variety of resumes and job descriptions, a few genuine problems surfaced that are worth being honest about.

4.1 The "Strict Recruiter" Heuristic

The most frustrating gap was between high semantic scores and what actual hiring managers would do. In several cases, a candidate scored well on alignment but would likely have been passed over in a real screening because of unstated requirements. One example that stuck with me: a senior role description listed fairly generic design principles, but the hiring manager apparently wanted someone specifically from a hyper-growth startup background. Nowhere in the posting did it say that. Elevate had no way to know. The pedigree graph helps partially - it can pick up on company type and infer something about the environments a candidate has worked in - but it's an imperfect fix for what is fundamentally a human communication problem.

4.2 Latency and Computational Overhead

The two-stage architecture also introduced latency that would be a real problem in production. The bi-encoder step is fast. But the cross-encoder — which has to process token-level relationships across paired texts — is noticeably slow on local hardware, especially when you're simulating high-volume screening. We haven't solved this yet. Model quantization and knowledge distillation are the obvious directions to explore, but We haven't had a chance to implement and test them rigorously. It's something that would need to be addressed before any serious deployment.

4.3 Data Messiness and Format Anomalies

PDF parsing is messier than it should be. pdfplumber handles a lot of cases well, but creative resume designs — floating text boxes, invisible tables, decorative timelines, multi-column layouts — can completely scramble the extracted text. In one case, a candidate's work history ended up merged with their skills section because of how the columns had been laid out. In another, a graphical timeline was extracted as a string of dates with no surrounding context. These failures are frustrating because the same design choices that make a resume visually memorable are exactly the ones that break the parser. There's a real tension there that We don't think has a clean solution at the text-extraction level.

5 Conclusions

Taking a step back, We think Elevate demonstrates something worth building on: that multi-model pipelines can make resume screening more transparent and more accurate than traditional keyword-based approaches. The semantic alignment scores capture meaning rather than wording. The seniority-aware weighting means the evaluation makes sense for the actual candidate being assessed. And the feedback generation gives both recruiters and applicants something actionable, rather than just a score with no explanation.

That said, the system has real limitations that I'd want to address in future work. The PDF parsing struggles with non-standard layouts, and integrating some form of computer vision to handle spatial structure would probably help a lot. The skill taxonomy and pedigree graph currently require manual updates, which doesn't scale - a self-supervised approach trained on large volumes of job postings would be more robust.

References

[1] Deshmukh, A., & Raut, A. (2025). Applying bert-based nlp for automated resume screening and candidate ranking. *Annals of Data Science*, 12(2), 591–603.

[2] Das, D. S., Nair, S. A., & Aneesh, P. (2025). AI resume analyzer: Smart resume evaluation and enhancement. *International Journal of Scientific Research in Engineering and Management*, 9(4), 1–9.

[3] Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

[4] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, 3982–3992.

[5] Rojas-Galeano, S., Posada, J., & Ordoñez, E. (2022). A bibliometric perspective on AI research for job-résumé matching. *The Scientific World Journal*, 2022, 8002363.